Logistic Regression

1. Introduction: Logistic Regression in Machine Learning

Logistic Regression is one of the most critical supervised learning algorithms in the field of Artificial Intelligence, specifically designed for binary classification problems. Unlike Linear Regression, which predicts continuous numerical values, Logistic Regression predicts the probability that a given input belongs to a specific category (e.g., "Pass" vs. "Fail" or "Spam" vs. "Not Spam").

1.1 The Limitation of Linear Regression

In early computational statistics, researchers attempted to use Linear Regression $y = wx + b$ for classification. However, this approach faced two major flaws:

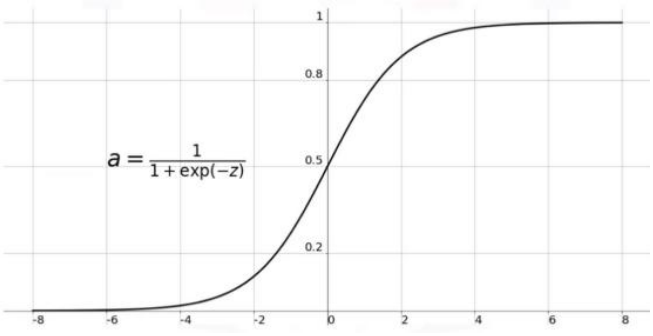
- **Sensitivity to Outliers**: A single extreme data point can significantly shift the regression line, ruining the classification accuracy.
- **Unbounded Outputs:** Linear regression can predict values like 1.5 or -0.2, which make no sense when representing probabilities

1.2 The Sigmoid Transformation

To solve these issues, Logistic Regression At its core, the algorithm performs a linear transformation of input features and then "squashes" the result through a non-linear **Sigmoid activation function**:

$$\hat{y}^{(i)} = \sigma(w^T x^{(i)} + b)$$

$$\sigma(z^{(i)}) = \frac{1}{1+e^{-z^{(i)}}}$$



1.3 Role in Modern AI

Logistic Regression is more than just a standalone algorithm; it is the computational building block of a Neural Network. A single neuron in a Deep Learning model is essentially a Logistic Regression unit. Understanding its optimization via **Gradient Descent** and the **Chain Rule** is prerequisite to mastering complex architectures like Transformers or CNNs.

1. The Hypothesis Function: From Linear to Logistic

The goal of the hypothesis function $\boldsymbol{\sigma(z)}$ is to take a set of input features $\boldsymbol{x}$ and output a probability $P(y = 1|\, x)$. This process happens in two distinct stages.

2.1 The Linear Predictor $\boldsymbol{z}$

First, the model calculates a weighted sum of the inputs, plus a bias term. This is identical to the equation for a straight line in Linear Regression:

$$z \;=\; w_0x_0 + w_1x_1 + w_2x_2 + \ldots + w_nx_n \;+ b$$

In vector notation, this is written as:

$$z \;= w^T x \;+\; b$$

Here, z can be any real number ($-\infty$ to ∞.) However, a probability must be between 0 and 1.

2.2 The Sigmoid (Logistic) Activation

To constrain z into a valid probability range, we pass it through the Sigmoid Function (also known as the Logistic Function). The mathematical form is :

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

2.3 The Final Hypothesis

By combining the linear equation and the Sigmoid function, we get the complete hypothesis for Logistic Regression:

$$\hat{y} = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}}$$

Interpretation of the Output:

- If $\hat{y} = 0.7$, the model is predicting a **70% probability** that the input belongs to Class1.
- Consequently, there is a **30% probability** (1 - 0.7) that it belongs to Class 0

3. The Decision Boundary: Defining the Class Separator

The hypothesis function $\boldsymbol{\sigma(z)}$ provides a probability $P(y = 1| x)$. To make a discrete prediction, we must choose a threshold. The standard threshold used in Logistic Regression is 0.5.

3.1 The Threshold Logic

The decision rule is defined as:

- **Predict** $\hat{y} = 1$ if $\boldsymbol{\sigma(z)} \geq 0.5$
- **Predict** $\hat{y} = 0$ if $\boldsymbol{\sigma(z)} < 0.5$

3.2 The Linear Equation of the Boundary

Because the Sigmoid function $\boldsymbol{\sigma(z)}$ equals exactly 0.5 when $z = 0$,the decision boundary occurs exactly when the linear part of our hypothesis equals zero:

$$z = w_0 x_0 + w_1 x_1 + w_2 x_2 + \ldots + w_n x_n + b = 0$$

This means the Decision Boundary is a linear function of the input features.

- In 2D space, the boundary is a straight line.
- In 3D space, the boundary is a flat plane.
- In higher dimensions, it is a hyperplane.

3.3 Linear vs. Non-Linear Boundaries

A standard Logistic Regression model can only separate data that is linearly separable. If the classes are mixed in a complex or circular way, a straight line will result in high error.

To solve this without moving to a complex Neural Network, researchers use Feature Mapping (Polynomial Features). By adding terms like x_1^2 or $x_1 x_2$ to the input vector, the model can create **non-linear boundaries**, such as circles or ellipses, while the underlying optimization math remains linear.

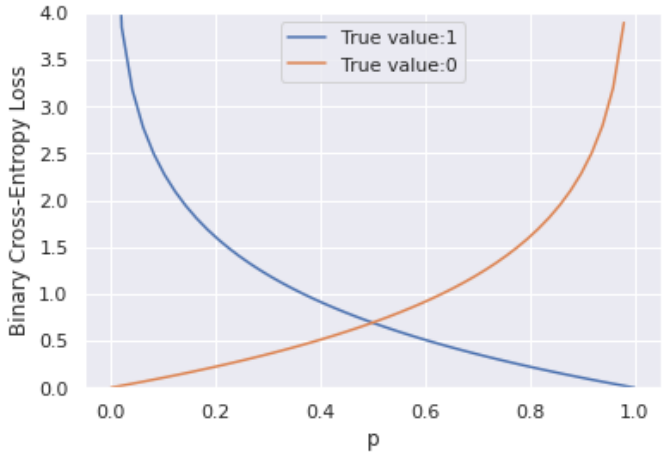
3. Logistic Regression Cost Function

In Logistic Regression, we seek a cost function that penalizes the model when it predicts a probability far from the actual label (y). While Mean Squared Error (MSE) is used in Linear Regression, it is unsuitable here because the non-linear Sigmoid function makes the MSE cost function **non-convex**, leading to multiple local minima that trap Gradient Descent. Instead, we use **Log Loss**, derived from the principle of Maximum Likelihood Estimation. For a single training example, the loss is defined as

$$L(\hat{y}^{(i)}, y^{(i)}) = \frac{1}{2}(\hat{y}^{(i)} - y^{(i)})^2$$

$$L(\hat{y}^{(i)}, y^{(i)}) = -(y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

- If $y^{(i)} = 1$: $L(\hat{y}^{(i)}, y^{(i)}) = -\log(\hat{y}^{(i)})$ where $\log(\hat{y}^{(i)})$ and $\hat{y}^{(i)}$ should be close to 1
- If $y^{(i)} = 0$: $L(\hat{y}^{(i)}, y^{(i)}) = -\log(1 - \hat{y}^{(i)})$ where $\log(1 - \hat{y}^{(i)})$ and $\hat{y}^{(i)}$ should be close to 0



The cost function is the average of the loss function of the entire training set. We are going to find the parameters w and b that minimize the overall cost function.

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m} L\left(\hat{y}^{(i)}, y^{(i)}\right) = -\frac{1}{m}\sum_{i=1}^{m}\left[\left(y^{(i)}\log\left(\hat{y}^{(i)}\right) + \left(1-y^{(i)}\right)\log\left(1-\hat{y}^{(i)}\right)\right]$$

The loss function measures how well the model is doing on the single training example, whereas **the cost function** measures how well the parameters w and b are doing on the entire training set.

The primary advantage of this cost function is that it is convex, ensuring a single global minimum. To minimize $\boldsymbol{J(w,b)}$ we calculate the partial derivative with respect to each weight w_j using Gradient Descent Algorithm

4. Optimization via Gradient Descent

Gradient Descent is an iterative optimization algorithm used to **minimize** the Cost Function $J(w,b)$. In Logistic Regression, because the cost function is convex, Gradient Descent is guaranteed to find the optimal weights that result in the highest classification accuracy. The Update Rule The algorithm updates the parameters w and b by moving them in the opposite direction of the gradient. For each iteration, the update is performed as follows starting with small random values for w and b :

$$\text{where } \; j \; = 0, 1, 2, 3, \ldots, n$$

Repeat until convergence {

$$w_j := \; w_j \; - \; \alpha \frac{\partial J(w,b)}{\partial w_j}$$

$$b := \; b \; - \; \alpha \frac{\partial J(w,b)}{\partial b} \; \}$$

For **Logistic Regression** we have :

Repeat until convergence {

$$w_j := w_j - \alpha \frac{1}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})\, x_j{}^{(i)}$$

$$b := b - \alpha \frac{1}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)}) \}$$

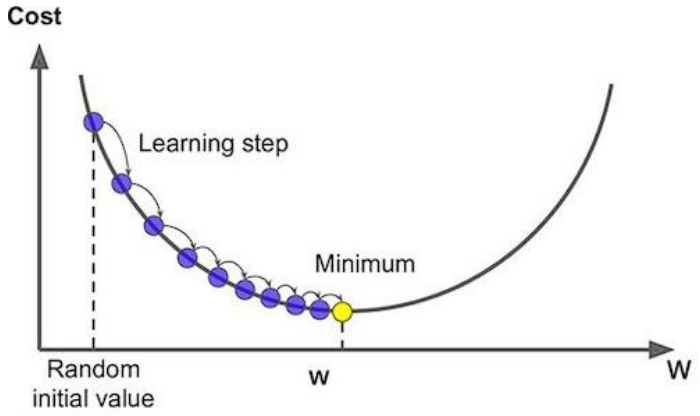
The Gradient ($\frac{\partial J(w,b)}{\partial w_j}$): As we derived using the **Chain Rule**, this is simply $(\hat{y}^{(i)} - y^{(i)})\, x_j{}^{(i)}$. It tells us the direction and steepness of the slope

The Learning Rate α This is a small positive number (e.g., 0.01 or \$0.001) that determines the size of the steps.

- If α is too **large,** the model might overshoot the minimum and fail to converge.
- If α is too **small,** the training will be extremely slow.

Convergence : The process repeats until the gradient is nearly zero, meaning the "slope" is flat and we have reached the bottom of the bowl. At this point, the weights w are optimized, and the Logistic Regression model is ready to make predictions on new data

The final solution obtained by Gradient Descent depends on the starting values of the parameters, but for convex cost functions, any starting point will converge to the global minimum



Let's simplify : assume that we have $J(w)$

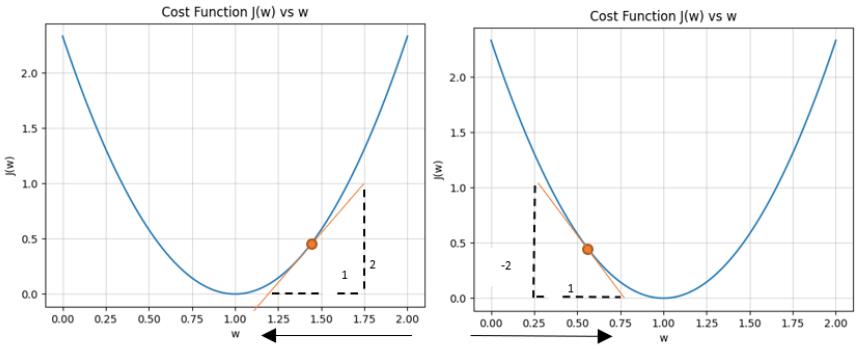
$$w_j := w_j - \alpha \frac{\partial J(w)}{\partial w_j}$$

$when \frac{\partial}{\partial w} J(w) > 0 \ this \ mean$

$$w = w - \alpha \,.(positive \ number\ ) \ so \ w \ will \ move \ to \ the \ \boldsymbol{left}$$

$when \ \frac{\partial}{\partial w} \ J(w) < 0 \ this \ mean$

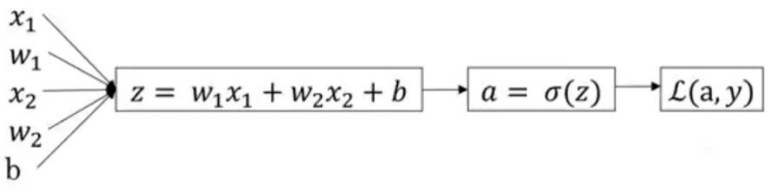
$$w = w - \alpha \,.(negaitve\ ) \ so \ w \ will \ move \ to \ the \ \boldsymbol{right}$$

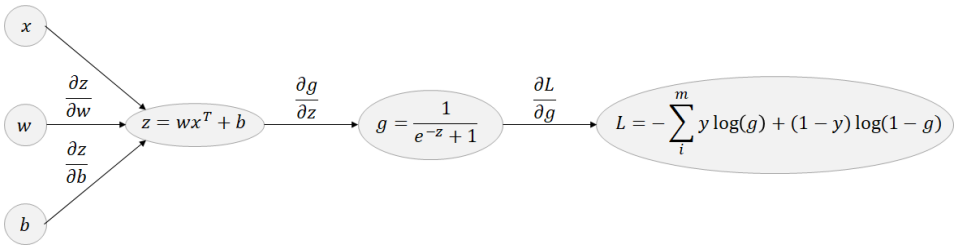


5. Chain Rule for Logistic Regression

We can prove the previous equation using **chain rule**

In Logistic Regression, the Chain Rule is the mechanism used to calculate the gradient of the Cost Function $J(w, b)$ with respect to the weights w Because Logistic Regression is a "nested" function, you cannot calculate the derivative of the weights in one step. You have to break it down into three specific "links" in the chain.





The Three Links of the Chain

To understand how a change in weight affects the final error, we look at the process in reverse:

- **Link 1:** How much does the Cost $J(w,b)$ change when the Prediction $a = \hat{y}$ changes?
- **Link 2:** How much does the Prediction $a = \hat{y}$ change when the Linear Sum z changes? (This is the derivative of the Sigmoid)
- **Link 3:** How much does the Linear Sum z change when the Weight w changes?

Mathematically, it looks like this:

$$\frac{\partial L(a,y)}{\partial w_j} = \frac{\partial L(a,y)}{\partial a} * \frac{\partial a}{\partial z} * \frac{\partial z}{\partial w_j}$$

And :

$$\frac{\partial L(a,y)}{\partial a} = \frac{a - y}{a(a - y)}, \qquad \frac{\partial a}{\partial z} = a(a - y), \qquad \frac{\partial z}{\partial w_j} = x_j$$

So, we get :

$$\frac{\partial L(a,y)}{\partial w_j} = (a - y)x_j$$

and for m training examples :

$$\frac{\partial J(w,b)}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}\frac{\partial L(a,y)}{\partial w_j}$$

$$\frac{\partial J(w,b)}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})\, x_j{}^{(i)}$$

6. Performance Evaluation: Beyond Accuracy

Once a Logistic Regression model is trained, we must evaluate its predictive power. Because classification involves two types of errors (False Positives and False Negatives), we use specific metrics to understand the model's behavior.

7.1 The Confusion Matrix

The Confusion Matrix is a $2\ x\ 2$ table that summarizes the prediction results. It is the foundation for all other metrics.

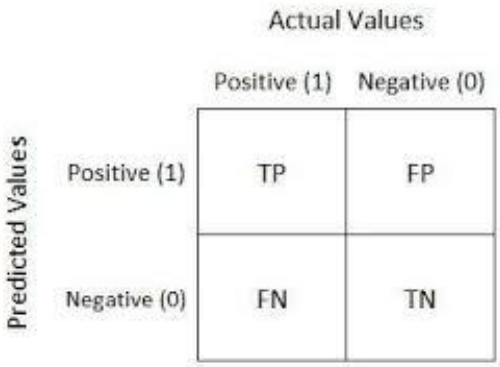
- True Positive (TP): Predicted 1, Actual 1.
- True Negative (TN): Predicted 0, Actual 0.
- False Positive (FP): Predicted 1, Actual 0 (Type I Error).
- False Negative (FN): Predicted 0, Actual 1 (Type II Error).

$$\text{Precision} = \frac{TP}{(TP + FP)}$$

$$\text{Recall} = \frac{TP}{(TP + FN)}$$

$$Accuracy\ = \frac{TP}{(TP + FP + FN + TN)}$$

$$F1 - Score\ =\ 2\ \frac{Precision\ *\ Recall}{Precision + Recall}$$



7.2 Key Metrics

While **Accuracy** is the most common metric, it fails if your dataset is imbalanced (e.g., 99% of samples are Class 0). Instead, we use:

1. **Precision:**
    - Meaning: Of all the times the model predicted "1", how many were actually "1"? (Focuses on avoiding False Positives).
2. **Recall (Sensitivity):**
    - Meaning: Of all the actual "1"s in the data, how many did the model find? (Focuses on avoiding False Negatives).
3. **F1-Score:**
    - Meaning: The harmonic means of Precision and Recall. This is the best single metric for imbalanced data.